

Тема № 13. Обработка особых (исключительных) ситуаций

13.1. Основы механизма обработки особых (исключительных) ситуаций

Определение особой (исключительной) ситуации:

Любая ситуация, достижимая в процессе выполнения программы, может быть объявлена программистом как особая или исключительная. Например, какая-то переменная приняла значения в заданном диапазоне (вышла из заданного диапазона), встретился конец файла, деление на 0, переполнение и т.д.

Следует отметить, что исключительной может быть объявлена синхронная ситуация, т.е. такая, которая может быть достигнута в процессе выполнения программы, прерывания от внешних устройств синхронными ситуациями не являются

Для реализации механизма обработки исключений в Си++ введены ключевые слова:

try (контролировать);
catch (ловить);
throw (генерировать).

Общий формат обработки исключительных ситуаций:

```
try // Контролируемый блок
{
    .....
    if (условие_определяющее_искл_ситуацию)
        throw значение_исключения; // Генерация исключения
    .....
}
// Один или несколько обработчиков исключений
catch(тип_исключения1 имя) { операторы }
.....
catch(тип_исключенияN имя) { операторы }
```

Механизм работы:

Как только выполняется оператор генерации исключения:

throw значение_исключения;

управление передается за пределы контролируемого блока в один из подходящих обработчиков исключений catch. Подходящий обработчик ищется по типу исключения. Обработчики исключений похожи на функцию с одним параметром, не возвращающую значение. После выполнения одного из обработчиков управление передается оператору, который первым следует за обработчиками, обратно в контролируемый блок управление не передается.

13.1.1. Пример обработки исключения внутри функции, в которой сгенерировано исключение

```
#include <iostream>
using namespace std;
double mydiv(double x, double y)
{
    // Обработка исключения внутри функции
    try // Контролируемый блок
    {
        if (y == 0) throw '0'; // Генерация исключения 1      тип char
        if (x < 0) throw (char *)"negative 1 param"; // Генерация исключения 2      тип
char *
        if (y < 0) throw (char *)"negative 2 param"; // Генерация исключения 3      тип
char *
        return x / y;
    }
    catch (char* str) // Обработчик исключения 2 или 3 через параметр передается значение
исключения
    {
        cout << "Exception: " << str << endl;
    }
    catch (char) // Обработчик исключения 1 для типа char
    {
        cout << "Exception: x=" << x << " y=" << y << endl;
    }
    return 0;
}
int main()
{
    cout << "Rez=" << mydiv(1, -2) << endl;
    return 0;
}
```

Будет напечатано:

Exception: negative 2 param

Rez=0

13.1.2. Пример обработки исключения вне функции, в которой сгенерировано исключение

```
#include <iostream>
using namespace std;
// В функции исключение генерируется но не обрабатывается
// обработка возложена на внешнюю функцию
double mydiv(double x, double y) // throw(char *, double) Могло быть так до стандарта C++17,
теперь исключено
{
    if (y == 0) throw (char *)"divide on 0";
    if (y < 0) throw y;
    if (x < 0) throw x;
    return x / y;
}
```

```

int main()
{
    double x = 1, y = 0;
    try
    {
        cout << "Rez=" << mydiv(x, y) << endl;
    }
    catch (char* str)
    {
        cout << "Exception: " << str << endl;
    }
    catch (double param)
    {
        cout << "Exception: " << "param=" << param << endl;
    }
    cout << "End program" << endl;
    return 0;
}

```

Будет напечатано в Visual C++ 2022:
Exception: divide on 0

End program

Будет напечатано в Code::Blocks v. 20.03:
Rez=Exception: divide on 0

End program

Стандарт C++11 содержит ключевое слово *noexcept* (аналог *throw()*). Означает, что функция не может генерировать не обработанное исключение:
void myfun() noexcept;

13.1.3. Пример генерации исключения как объекта класса

В этом случае исключение может быть более информативным, дополнительная информация сохраняется в полях класса.

```

#include <iostream>
#include <string>
using namespace std;

struct my_data // Класс для задания исключения при создании в полях хранится дополнительная информация об исключении
{
    double x, y;
    string str;
    my_data(double a, double b, string s)
    {
        x = a; y = b; str=s;
    }
};

double mydiv(double x, double y)
{

```

```

        if (y == 0) throw my_data(x, y, "Divide on 0"); // Создаем исключение как объект класса
        if (y < 0 || x < 0) throw my_data(x, y, "Negative parameter"); // Создаем исключение
        как объект класса
        return x / y;
    }

    int main()
    {

        try
        {

            cout << "Rez=" << mydiv(5, 0) << endl;
        }
        catch (my_data& d)
        {
            cout << d.str << " x=" << d.x << " y=" << d.y << endl;
        }

        return 0;
    }

```

Будет напечатано в Visual C++ 2022:
Divide on 0 x=5 y=0

Будет напечатано в Code::Blocks v. 20.03:
Rez=Divide on 0 x=5 y=0

Основная идея:

Функция, сталкивающаяся с неразрешимой проблемой, формирует исключение в надежде на то, что вызывающая ее (прямо или косвенно) функция сможет обработать исключение.

13.2. Формы обработчиков исключений

После блока try может быть несколько обработчиков исключений, существует три различные формы:

```

catch(min_искл имя_искл) {.....}
catch(min_исключения) {.....}
/* не предусматривает использование значения исключения, важен тип */
catch(...) {.....}
// Подходит для исключения любого типа
// обработчик должен быть последним
#include <iostream>
using namespace std;
class ZeroDivide {};
class Overflow {};
float mydiv(float x, float y)
{
    if (y == 0) throw ZeroDivide();
    double z = x / y;
    if (z > 1e+30) throw Overflow();
    if (z == 0) throw 1; // Исключение типа int (Сработает универсальный обработчик)
    return z;
}

```

```

int main()
{
    try
    {
        cout << "Rez=" << mydiv(1e20, 1e-20) << endl;
    }
    catch (Overflow) // Важен тип исключения имя параметра отсутствует
    {
        cout << "Overflow\n";
    }
    catch (ZeroDivide)
    {
        cout << "ZeroDivide\n"; // Важен тип исключения имя параметра отсутствует
    }
    catch (...) // Обработчик без типа подходит для любого исключения должен быть
последним
    {
        cout << "Exception\n";
    }
    return 0;
}

```

Будет напечатано в Visual C++ 2022:
Overflow

Будет напечатано в Code::Blocks v. 20.03:
Rez=Overflow

Сравнения по типам в обработчиках имеет более широкий смысл, например, обработчик `catch(T t) {.....}` подходит для обработки исключений типа: `const T`, `T&`, `const T&`, и типов производных классов от `T`.

13.3. Формы выражений генерации исключений

2 формы выражений генерации исключений:

throw выражение;

/ Исключение формируется как статический объект, значение которого определяется выражением генерации */*

```

throw;
#include <iostream>
using namespace std;
double mydiv(double x, double y)
{
    try
    {
        if (y == 0) throw "ZeroDivide"; // Тип исключения char *
        if (y < 0) throw y; // Тип исключения double
        return x / y;
    }
    catch (char* str)
    {
        cout << str << endl;
    }
    catch (double y)
    {

```

```

        throw; // Ретрансляция исключения во внешний блок
    }
    return 0;
}

int main()
{
    try // Внешний блок try
    {
        cout << "Rez=" << mydiv(9, -4) << endl;
    }
    catch (double a) // Сюда передается исключение из внутреннего блока
    {
        cout << "Exception: a=" << a << endl;
    }
    return 0;
}

```

Будет напечатано в Visual C++ 2022:
Exception: a=-4

Будет напечатано в Code::Blocks v. 20.03:
Rez=Exception: a=-4

Оператор «ретранслирует» уже существующее исключение в блок try верхнего уровня, используется внутри блока catch в случае вложенных блоков try.

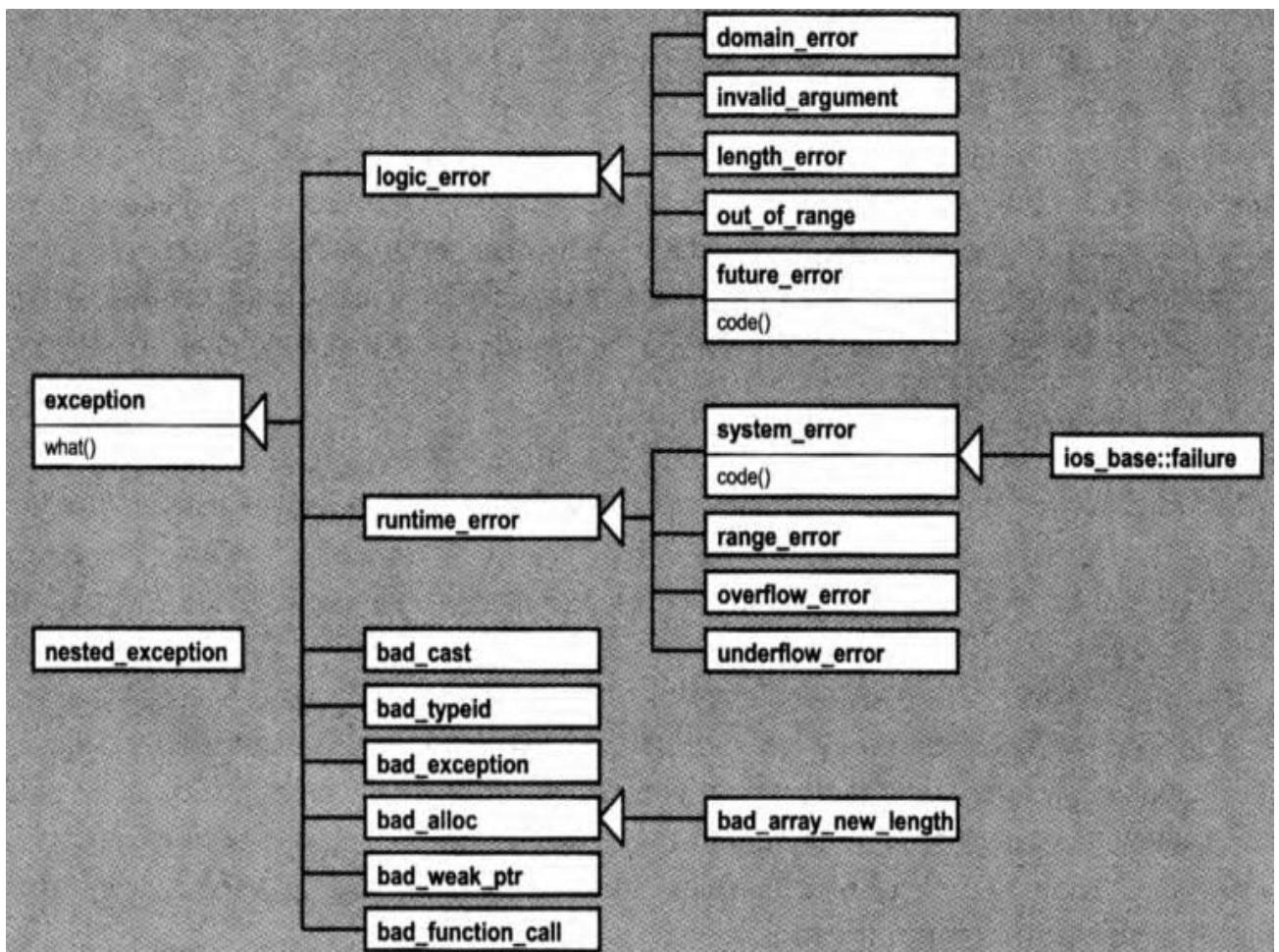
При вложении контролируемых блоков исключение, возникшее во внутреннем блоке, последовательно «просматривает» обработчики, переходя от внутреннего блока к внешнему, до тех пор, пока не будет найден подходящий обработчик. Если во всей совокупности обработчиков не будет найден подходящий, то выполняется аварийное завершение программы

13.4. Обработка исключений в стандартной библиотеке Си++

Все исключения являются производными от класса exception (пространство имен std, заголовочный файл exception). Класс содержит строку текста с описанием исключения (char *), есть конструктор для инициализации этой строки в производных классах от exception, в классе exception строка содержит «std::exception», чтобы получить указатель на строку, вызывается метод char * what();

Исключения можно разделить на 3 категории:

- языковая поддержка;
- логические ошибки;
- ошибки времени выполнения.



```
#include <iostream>
#include <exception>
using namespace std;
```

```
struct mydata : public exception // Класс для задания исключения при создании в полях хранится
дополнительная информация об исключении
{
```

```
    double x, y;
    mydata(double a, double b) : exception()
    {
        x = a; y = b;
    }
};
```

```
double div(double x, double y)
{
```

```
    if (y == 0) throw exception(); // Исключение класса exception
    if (y < 0 || x < 0) throw mydata(x, y); // Исключение класса производного от exception
    return x / y;
}
```

```
int main()
{
```

```
    try
    {
```

```
        cout << "Rez=" << div(5., -5.) << endl;
```

```

    }
    catch (mydata& d) // Обработчик класса производного от exception
    {
        cout << d.what() << " x=" << d.x << " y=" << d.y << endl;
    }
    catch (exception& d) // Обработчик класса exception
    {
        cout << d.what() << endl;
    }
    return 0;
}

```

Будет напечатано в Visual C++ 2022:
Unknown exception x=5 y=-5

Будет напечатано в Code::Blocks v. 20.03:
Rez=std::exception x=5 y=-5

Все функции в определении класса exception имеют пустую спецификацию поехсерт (или ранее throw()).

Исключения языковой поддержки используются на уровне языка C++. Эти исключения генерируются при неудачных попытках выполнения некоторых операций. Некоторые классы:

- исключение класса bad_alloc генерируется при неудачном выполнении глобального оператора new (кроме версии new с запретом исключений);
- исключение класса bad_cast генерируется оператором dynamic_cast, если преобразование типа по ссылке во время выполнения завершается неудачей;
- исключение класса bad_typeid генерируется оператором typeid, предназначенным для идентификации типов во время выполнения, если аргументом typeid является ноль или null-указатель, генерируется исключение;
- исключение класса bad_exception предназначено для обработки непредвиденных исключений.

Логические ошибки обусловлены нарушением внутренней логики программы. Предполагается, что их можно найти и предотвратить еще до начала выполнения программы. Базовый класс для логических ошибок:

```
class logic_error : public exception
```

В стандартной библиотеке определены следующие классы для логических ошибок (заголовки классов):

```

class invalid_argument : public logic_error // неверный аргумент

class out_of_range : public logic_error // вне диапазона

class length_error : public logic_error // неверная длина

class domain_error : public logic_error // вне допустимой области

```

Ошибки времени выполнения связаны с событием, с самой программой не связанным. Предполагается, что их нельзя обнаружить, пока программа не начала работать. Базовый класс:

```
class runtime_error : public exception // ошибка времени выполнения
```

Классы ошибок времени выполнения:

```
class range_error : public runtime_error // ошибка диапазона
```

Функция может возбудить исключение `range_error`, чтобы сообщить об ошибке во внутренних вычислениях.

```
class overflow_error : public runtime_error // переполнение
```

```
class underflow_error : public runtime_error { // потеря значимости
```

13.5. Структурная обработка исключений (факультативно)

Исключения в C++ это механизм обработки ошибок. Исключения представлены ключевыми словами `try/catch`. В мире Windows (для средств разработки Microsoft) существует еще один тип исключений: SE (Structured Exception) – структурированные исключения.

SEH (structured exception handling) введена в Windows для повышения надежность программного обеспечения. Компилятор генерирует специальный код на входах и выходах блоков исключений, создает таблицы вспомогательных структур данных для поддержки SEH.

С точки зрения программы, обработка обоих видов исключений – одинаковая. В компиляторе MS VS есть ключевые слова `__try/__except/__finally`. Они позволяют полностью выполнять обработку структурированных исключений способом, похожим на обработку C++ исключений.

13.5.1. Обработчики завершений

```
__try {  
    // Защищенный блок  
    ....  
}  
__finally {  
    // Обработчик завершения  
    ....  
}
```

Блок `__finally` будет выполнен в любом случае. Как при нормальном, так и не нормальном завершении.

13.5.2. Фильтры и обработчики исключений

```
__try {  
    // Защищенный блок  
    ...  
}  
__except(фильтр_исключений) { Синтаксис обработчика исключений:  
  
    // Обработчик исключений  
    ...  
}  
(За __try всегда 1 блок либо finally, либо except)
```

Возможные значения фильтров определены в заголовочном файле `Excpt.h`:

`EXCEPTION_EXECUTE_HANDLER` (исключение во время выполнения программы) при этом выполняется, так называемая «глобальная раскрутка» программы и выполняется код обработчика.

EXCEPTION_CONTINUE_EXECUTION –повторно проверяется фильтр. На первый взгляд, получим бесконечный процесс постоянно проверяется фильтр.

Для разрешения такой проблемы используют функции-фильтры, которые возвращают разные значения. Эта функция пишется в качестве выражения в блоке __except.

EXCEPTION_CONTINUE_SEARCH – система переходит к предыдущему блоку __try (верхнего уровня) и вычисляет его фильтр, код обработчика исключений не выполняется.

// MySEN_Simple.cpp : Defines the entry point for the console application.

//

#include <iostream>

#include <Excpt.h>

using namespace std;

int main()

{

int x;

int* p = 0;

while (1)

{

__try

{

*p = 100; // Здесь будет ошибка (обращение по нулевому адресу), которую

можно исправить

cout << "*p=" << *p << endl;

break;

}

__except (EXCEPTION_EXECUTE_HANDLER)

{

cout << "EXCEPTION_EXECUTE_HANDLER" << endl;

p = &x; // Исправляем ошибку

}

}

/*__finally

{

cout<<"Finally"<<endl;

*/

cout << "End Program" << endl;

return 0;

}